

CSE 4810 : Algorithm Engineering Lab

Lab - 1

MD ATIQUR RAHMAN

Group - CSE 2B

May 27, 2025

Task 1

Given an array `arr[]` of integers, the task is to find the Next Greater Element for each element of the array in order of their appearance in the array. You need to give a solution with $O(n)$ time and $O(n)$ space complexity.

Note: The Next Greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider the next greater element as -1 .

Input: `arr[] = [1, 3, 2, 4]`

Output: `[3, 4, 4, -1]`

Input: `arr[] = [6, 8, 0, 1, 3]`

Output: `[8, -1, 1, 3, -1]`

Task 2

Implement Stack using two Queues (`q1` and `q2`) following the below algorithm.

- Steps for `push(x)`:
 - Enqueue x to `q2`.
 - One by one dequeue everything from `q1` and enqueue to `q2`.
 - Swap the queues of `q1` and `q2`.
- Steps for `pop()`:
 - Dequeue an item from `q1` and return it.

Task 3

Given an input stream s consisting solely of lowercase letters, you are required to identify which character has appeared only once in the stream up to each point. If there are multiple characters that have appeared only once, return the one that first appeared. If no character has appeared only once, append '#' to the result. You need to solve it within $O(n)$ memory and time complexity.

Input: `s = "aabc"`

Output: `"a#bb"`

Explanation:

For every ith character we will consider the string from index 0 till index i first non repeating character is as follow- "a" - first non-repeating character is 'a' "aa" - no non-repeating character so '#' "aab" - first non-repeating character is 'b' "aabc" - there are two non repeating characters 'b' and 'c', first non-repeating character is 'b' because 'b' comes before 'c' in the stream.

Input: s = "bb"

Output: "b#"

Explanation:

For every character first non repeating character is as follow- "b" - first non-repeating character is 'b' "bb" - no non-repeating character so '#'

Helping Code

Implementation of Stack

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        """Push an item onto the stack."""
        self.items.append(item)

    def pop(self):
        """Pop an item off the stack. Returns None if the stack is empty."""
        if not self.empty():
            return self.items.pop()
        else:
            return None

    def empty(self):
        """Check if the stack is empty."""
        return len(self.items) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self.items)

# Example usage:
if __name__ == "__main__":
    s = Stack()
    s.push(10)
    s.push(20)
    print("Size:", s.size()) # Output: 2
    print("Pop:", s.pop())   # Output: 20
    print("Empty:", s.empty()) # Output: False
    s.pop()
    print("Empty after popping all:", s.empty()) # Output: True
```

Implementation of Deque

If you can access collections module. In Python 3.12.6 you can.

```

from collections import deque

# Create a deque
dq = deque()

# Add to the right (like queue)
dq.append(1)
dq.append(2)

# Add to the left
dq.appendleft(0)

# Remove from the right
dq.pop() # Removes 2

# Remove from the left
dq.popleft() # Removes 0

# Check if empty
print(len(dq) == 0)

# View the deque
print(dq) # deque([1])

```

If you can not then use the following implementation. Remember the following will cost you $O(n)$ for some operation while the above implementation will only cost you $O(1)$.

```

class Deque:
    def __init__(self):
        self.items = []

    def append(self, item):
        """Add to the right end."""
        self.items.append(item)

    def appendleft(self, item):
        """Add to the left end."""
        self.items.insert(0, item)

    def pop(self):
        """Remove and return item from right end."""
        if not self.empty():
            return self.items.pop()
        return None

    def popleft(self):
        """Remove and return item from left end."""
        if not self.empty():
            return self.items.pop(0)
        return None

    def empty(self):
        """Check if deque is empty."""

```

```

        return len(self.items) == 0

    def size(self):
        """Return the number of items."""
        return len(self.items)

    def __str__(self):
        return str(self.items)

# Example usage
dq = Deque()
dq.append(10)
dq.appendleft(5)
dq.append(15)
print(dq)           # Output: [5, 10, 15]
print(dq.popleft()) # Output: 5
print(dq.pop())     # Output: 15
print(dq)           # Output: [10]

```